

Reviewer Pack — Minimal Polynomial Hierarchy Depth of Monotone DNF Formulas ...

Entry 361f53b2014d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 13:30:15 UTC

Minimal Polynomial Hierarchy Depth of Monotone DNF Formulas Bounds k-CLIQUE Circuit Size

Entry ID: 361f53b2014d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 13:30:15 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics **Field B** (complexity object): Monotone Circuit Complexity

Statement:

[‘For any monotone DNF formula representing the k-CLIQUE problem, its minimal polynomial hierarchy depth is upper-bounded by $n^{k/2}$.’, ‘The polynomial hierarchy depth of a monotone DNF formula F for k-CLIQUE is defined as the smallest level m such that there exists a polynomial-time computable function f mapping $\{0,1\}^m$ to $\{0,1\}$ that computes the truth table of F .’, ‘For any k and $n \leq 40$, there exists an instance of the k-CLIQUE problem for which its monotone DNF formula has polynomial hierarchy depth at most $n^{k/2}$.’]

Rationale (proposer’s reasoning):

[‘Minimal polynomials in algebraic combinatorics often capture essential structural properties that can be used to prove circuit lower bounds.’, ‘The polynomial hierarchy is a natural measure of computational difficulty, and bounding its depth for monotone DNF formulas could potentially lead to nontrivial lower bounds on the complexity of k-CLIQUE.’, ‘Minimal polynomials are computationally accessible, making this conjecture testable within the constraints.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7eb69d50222bbc6d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across all 30 random seeds and for all $k \leq 40$, the average minimal polynomial hierarchy depth of monotone DNF formulas representing the k-CLIQUE problem does not exceed $n^{k/2}$ with a standard deviation of no more than $0.1n^k$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "monotone DNF formulas" AND minimal polynomial hierarchy depth" - "polynomial hierarchy depth" AND k-CLIQUE problem" - "algebraic combinatorics" AND monotone circuit complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_monotone_dnf(n, k):
        # Generate a random monotone DNF formula for k-CLIQUE with n variables
        clauses = []
        for _ in range(k):
            clause = set()
            while len(clause) < n:
                lit = random.randint(0, 2 * n - 1)
                if lit % 2 == 0 and lit // 2 not in clause:
                    clause.add(lit // 2)
            clauses.append(clause)
        return clauses

    def truth_table(dnf):
        # Compute the truth table of a monotone DNF formula
        n = len(dnf) * 2
        tt = []
        for i in range(1 << n):
            assignment = [(i >> j) & 1 for j in range(n)]
            value = any(all(assignment[lit // 2] if lit % 2 == 0 else not assignment[lit // 2] for
            tt.append(value)
        return tt

    def polynomial_hierarchy_depth(tt):
        # Compute the minimal polynomial hierarchy depth of a monotone DNF formula
```

```

n = len(tt)
m = 1
while True:
    f = [0] * (2 ** m)
    for i in range(2 ** m):
        f[i] = tt[sum((i >> j) & 1 << j for j in range(m))]
    if all(f[i] == f[i ^ (1 << j)] for j in range(m)):
        break
    m += 1
return m

n_max = 40
k_max = 40
instances_tested = 0
total_depth = 0

for n in [5, 10, 15, 20, 30, 40]:
    for k in range(1, min(k_max + 1, n + 1)):
        dnf = generate_monotone_dnf(n, k)
        tt = truth_table(dnf)
        depth = polynomial_hierarchy_depth(tt)
        total_depth += depth
        instances_tested += 1

conjecture_holds = total_depth <= (n_max ** k_max / 2) * instances_tested
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "polynomial_hierarchy_depth",
    "metric_value": total_depth,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_depth = sum(r["metric_value"] for r in results) / len(results)
    std_depth = math.sqrt(sum((r["metric_value"] - mean_depth) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_depth} std={std_depth} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: Run the test again with increased time limits or consider alternative methods to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15300
2	preregistration	ollama_remote	glm4:latest	0	0	9492
3	novelty	ollama_remote	glm4:latest	0	0	8561
4	novelty	ollama_remote	glm4:latest	0	0	9000
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13150
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9761
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12365
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11502
9	judge	ollama_remote	glm4:latest	0	0	11953

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 101084 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/361f53b2014d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/361f53b2014d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/361f53b2014d.tar.gz` (if generated)